010

# Page Replacement

Operating Systems

# Page Replacement

**Objective**

You will implement a **program** (normal user-space simulation) that reads a **reference string**—a sequence of **virtual page numbers** that a process accesses—and reports how many **page hits** and **page faults (misses)** occur for a **fixed number of physical frames** when you apply three classic **replacement policies**:

- **FIFO** (First-In, First-Out)
- **MIN** (also called **Optimal** or **OPT**—replace the page used farthest in the future)
- **LRU** (Least Recently Used)

The goal is to see how the **same** access pattern behaves under different algorithms

## Scenario: Limited physical frames

The OS has **N** page **frames** for this process (capacity **N ≥ 1**). At any time at most **N** distinct **page numbers** may reside in memory. When the CPU references a page:

- If that page is **already** in one of the frames → **hit** (no replacement).
- If it is **not** in memory → **miss** / **page fault**: you must bring the page in.
    - If fewer than **N** frames are in use, install the page in a free slot.
    - If all **N** frames already hold pages, **evict** exactly one page according to the active policy, then load the referenced page.

This lab does **not** model disk latency, dirty bits, or write-back. Only **reference order** and **hit/miss counting** matter.

## Assignment Tasks

### Task 1 — Input and validation

Your program must obtain:

1. **N** — number of frames (integer **≥ 1**).
2. A **reference sequence** — ordered list of page ids.

Allowed input styles (pick **at least one** and document it):

- **CLI:** e.g. N then a single string of space-separated page numbers, or **N + path to a text file** with one page per line.
- **Interactive:** prompt for **N**, then a line (or multiple lines) of page numbers.

**Validation:** reject empty sequences, invalid **N**, or non-numeric tokens with a clear error message. If you allow negative page ids, say so; otherwise reject them.

## Task 2 — Run three algorithms on the same input

For the **same N** and **same** reference string, simulate in order:

3. **FIFO**
    - Treat loaded pages as a **queue** in order of **arrival** into memory (when first loaded).
    - On eviction, remove the **oldest by arrival time** among the **N** residents.
4. **MIN / Optimal**
    - On eviction when memory is full, among pages **currently resident**, remove the one whose **next** appearance in the **remaining** reference string (strictly **after** the current step) occurs **farthest** in the future.
    - If a resident page **never** appears again, it is always a candidate for eviction; if **several** never appear again, define a deterministic tie-break (e.g. **smallest page id**) and document it.
    - If multiple pages share the **same farthest-next** index, use the **same** documented tie-break.
5. **LRU**
    - On eviction when memory is full, remove the page whose **last access** (most recent prior reference to that page in the sequence **so far**) is **least recent** (oldest last-use).
    - The **current** reference counts as "used" **after** you decide hit/miss (i.e. you update recency when the page is accessed).
    - Tie-break for equal last-use time: document (e.g. **smallest page id**).

**Output (required):** for **each** algorithm print at least:

- Algorithm name
- Total **hits** and total **misses**
- Optional but recommended: **hit rate** = hits / (hits + misses) as a percentage with two decimals

### Task 3 — On-screen trace (required for grading)

For **each** algorithm, print a **step-by-step** trace so the state of memory is visible. Minimum acceptable format:

- One line **per reference** (in order), showing: index or step number, **referenced page**, whether **HIT** or **MISS**, and the **contents of the N frames** after handling that reference (use _ or **-** for empty slots if you allow partial fill before **N** is reached).
- On a **miss** that causes **replacement**, show **which page was evicted** (and by which policy if not obvious).

A **comparison summary** block at the end (table: FIFO / MIN / LRU with hits and misses) is encouraged.

Example **shape** (your layout may vary):

```
FIFO (N=3)
 step  ref  result  frames[0..N-1]  victim
  1    7   MISS    [ 7, _, _ ]
  2    0   MISS    [ 7, 0, _ ]
...

Totals: hits=.. misses=..
```

Students may use ASCII tables or monospace columns; readability matters more than fancy graphics.

## Reference dataset

Use **exactly** this reference string and frame count so TAs can compare shapes (your **intermediate** frames may differ if you use a different tie-break—then document tie-breaks to match).

| Setting | Value |
|---|---|
| **Frame count N** | **3** |
| **Reference sequence** (in order) | **7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1** |

**Sanity check (not a substitute for your trace):** with **N = 3**, the sequence is long enough that every algorithm will show a mix of hits and misses; **MIN** typically achieves the **fewest** misses on this same string among the three (that is a known property of the optimal policy on a fixed trace—your counts should reflect that **if** your MIN matches the spec above).

## Extra points (optional)

- Add **Second Chance** or **Clock** as a fourth policy and compare to FIFO on the same input.
- Implement **Belady's anomaly** demo: show a sequence where **increasing N increases** misses for FIFO (document the classic example).